

**UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJE Y COMPILADORES (2026-I)**

**INFORME DE INVESTIGACIÓN: ESTUDIO COMPARATIVO
DE PARADIGMAS, LENGUAJES Y DISEÑO DE DSL**

Asignación II: Los Lenguajes de Programación

Autores:

Fernando Centeno (C.I. 30.810.484)
Juan Longart (C.I. 31.882.343)
Adrian Reina (C.I. 31.317.970)
Rafael Rodríguez (C.I. 31.882.367)

Profesor:

Félix Márquez
fmarquez@e.uneg.edu.ve

Ciudad Guayana, Mayo de 2026

1. Introducción

El diseño y análisis de los lenguajes de programación constituyen la base de la ciencia de la computación. En las etapas iniciales de formación, los estudiantes interactúan con los lenguajes principalmente como herramientas estáticas desde la perspectiva de programador aplicativo. Sin embargo, en el diseño de compiladores es fundamental analizar el lenguaje como un producto de software meticulosamente diseñado, con una arquitectura y restricciones formales rigurosas.

Este informe expone el estudio y resolución de la Asignación II, abordando el análisis crítico de paradigmas de programación clásicos y emergentes, la evaluación empírica (benchmarking) y de bajo nivel (léxico-sintáctico) de cuatro lenguajes modernos (Zig, Python, Rust, JavaScript), y el diseño abstracto de un Lenguaje de Dominio Específico (DSL) denominado 'Lenguaje L' para la operación de un sistema crítico de microredes eléctricas inteligente denominado ECO-GRID.

2. Actividad I: Matriz Descriptiva y Análisis de Paradigmas

El desarrollo de software contemporáneo está marcado por la convergencia multiparadigma. A continuación se detallan las características fundamentales de los paradigmas clásicos y emergentes:

- * **Paradigma Imperativo/Estructural:** Se fundamenta en la secuenciación de instrucciones explícitas y la gestión directa del estado del sistema y de la memoria. La mutabilidad y los efectos secundarios son centrales en este enfoque (ejemplos clásicos: C, Pascal; y modernos: Zig).

- * **Paradigma Orientado a Objetos (POO):** Estructura el software en términos de objetos que combinan datos y comportamiento. Los conceptos de encapsulamiento, herencia, composición y polimorfismo proveen abstracción y modularidad (ejemplos: Java, C++).

- * **Paradigma Funcional:** Trata la computación como la evaluación de funciones matemáticas y evita el estado mutable. Se caracteriza por funciones como ciudadanos de primer orden, inmutabilidad de datos, evaluación perezosa (lazy evaluation) y transparencia referencial (ejemplos: Haskell, Lisp).

- * **Paradigma Lógico/Declarativo:** Basado en la lógica matemática y relaciones. El programador define las reglas, hechos y metas, y el motor de inferencia unifica variables y resuelve cláusulas de Horn, abstrayendo el flujo de control (ejemplo clásico: Prolog).

- * **Paradigma Concurrente/Actores (Emergente):** Orientado al procesamiento paralelo y la distribución de carga. El modelo de actores evita compartir estado mediante el paso de mensajes aislados, previniendo condiciones de carrera a nivel lingüístico (ejemplos: Erlang, Elixir).

3. Actividad II: Estudio Comparativo de Lenguajes y Benchmarking

3.1. Análisis Morfológico (Léxico) de los Lenguajes

El análisis morfológico define cómo cada lenguaje agrupa los caracteres en tokens. Los detalles de los lenguajes analizados son:

- * **Python:** Utiliza indentación significativa (delimitación implícita de bloques por espacios en blanco) en lugar de llaves. Soporta palabras reservadas (`def`, `class`, `if`, `elif`). Sus identificadores son sensibles a mayúsculas y no pueden comenzar con números. Trata comentarios mediante el carácter `#`.

- * **JavaScript (Node.js):** Usa llaves `{}` para delimitación de bloques y punto y coma `;` opcional. Sus identificadores siguen la regla estándar `camelCase`. Soporta comentarios de una línea `//` y multilínea `/* ... */`. Es dinámicamente tipado.

- * **Rust:** Posee un analizador léxico estricto basado en llaves para delimitación de bloques. Las palabras reservadas son inmutables (`fn`, `let`, `mut`, `match`). La notación estándar de identificadores es `snake_case`. Soporta literales de bytes, caracteres y cadenas crudas.

- * **Zig:** Usa delimitación por llaves. Destaca por no tener variables ocultas y no permitir la asignación de variables sin inicialización previa. Las palabras reservadas incluyen `'fn'`, `'pub'`, `'const'`, `'var'`, `'comptime'`.

3.2. Análisis Sintáctico de Estructuras de Control

El análisis sintáctico valida la estructura gramatical del código. En Python, la estructura condicional se define mediante bloques indentados (`if/elif/else`). En JavaScript, Rust y Zig se emplean paréntesis para las condiciones y llaves para los bloques. Rust y Zig incorporan expresiones de control de flujo donde las estructuras condicionales y de coincidencia (`match`) pueden retornar valores directamente.

3.3. Arquitectura y Modelos de Ejecución Detallados por Lenguaje

* **Zig (Modelo de Memoria y Comptime):** Control de memoria explícito obligando a pasar alocadores (GeneralPurposeAllocator, ArenaAllocator) para evitar alocações ocultas. El compilador LLVM procesa la metaprogramación estática con 'comptime' en tiempo de compilación. Errores tratados como valores explícitos en uniones.

* **Rust (Borrow Checker y Propiedad):** Seguridad de memoria estática garantizada en compilación por el Borrow Checker mediante Ownership, Borrowing y Lifetimes sin Garbage Collector. Distingue entre bloques seguros y bloques 'unsafe' para accesos de bajo nivel. Compila a MIR y luego a LLVM IR.

* **Python (Máquina Virtual y GIL):** CPython compila a bytecode (.pyc) y lo ejecuta en una VM de pila. La estructura genérica PyObject añade sobrecarga por tipado dinámico. Gestión de memoria por conteo de referencias y Garbage Collector generacional cíclico. El cerrojo GIL impide la ejecución multihilo paralela en CPU.

* **JavaScript (Compilación JIT y Event Loop):** V8 utiliza una infraestructura JIT de dos etapas: Ignition (intérprete) y TurboFan (compilador JIT de código nativo optimizado con des-optimización dinámica). Memoria por recolector generacional (Scavenger + Mark-Sweep-Compact). Event Loop asíncrono con libuv sobre un único hilo nativo de JS.

3.4. Resultados del Benchmarking (Algoritmo de Collatz)

Para evaluar el rendimiento empírico de estos lenguajes, se implementó el cálculo iterativo de la conjetura de Collatz para todos los números desde 1 hasta N. A continuación se presentan los resultados obtenidos en el hardware local:

Lenguaje	Mecanismo Ejecución	Tiempo Promedio (ms)	Memoria Pico (MB)	Velocidad Rel.
Python	Interpretado (CPython / VM)	151557.35 ms	0.0003 MB	1.0x
JavaScript	JIT (Just-In Time) / V8 Engine	2401.99 ms	40.9648 MB	63.1x
Rust	Compilación Nativa (LLVM)	1848.26 ms	1.2500 MB	82.0x
Zig	Compilación Nativa (LLVM)	1981.14 ms	1.1000 MB	76.5x

Análisis del Benchmark: Los resultados evidencian la enorme diferencia entre los lenguajes compilados nativamente (Rust y Zig), que completan la tarea en milisegundos gracias al diseño optimizado de bucles por LLVM, frente a Python, que sufre penalizaciones por la sobrecarga del bucle de evaluación e introspección de tipos en su máquina virtual.

4. Actividad III: Diseño de un DSL (Lenguaje L) para ECO-GRID

El Lenguaje L es un DSL específico para operadores de microrredes eléctricas críticas. Se define formalmente a continuación:

4.1. Gramática EBNF Simplificada

```
<programa> ::= <sentencia>+
<sentencia> ::= <asignacion> | <llamada_fun> | <si_verdadero> | <mientras>
<asignacion> ::= <identificador> ":" <expresion> ";"
<llamada_fun> ::= <identificador> "(" <argumentos>? ")" ";"
<si_verdadero> ::= "si_verdadero" <condicion> "entonces" <sentencia>+ "fin_si"
<mientras> ::= "mientras" <condicion> "ejecutar" <sentencia>+ "fin_mientras"
<condicion> ::= <expresion> ("==" | ">" | "<" | ">=" | "<=") <expresion>
<expresion> ::= <identificador> | <numero> | <booleano>
```

4.2. Escenario A: Prevención de Fuga Térmica

```
init_grid;
mientras verdadero ejecutar
    temp := leer_temperatura(1); // Sensor en Batería Principal
    si_verdadero temp > 55 entonces
        conmutar_linea(5, 1); // Activar refrigeración auxiliar
        conmutar_linea(4, 0); // Desconectar paneles solares (carga térmica)
        conmutar_linea(1, 0); // Aislar consumo industrial
        conmutar_linea(2, 1); // Activar red comercial de respaldo
    fin_si
fin_mientras
```

4.3. Escenario B: Balance de Carga Inteligente

```
init_grid;
mientras verdadero ejecutar
    carga := estado_carga(1);
    paneles_gen := leer_generacion_solar();
    demanda := leer_demanda_interna();

    si_verdadero carga > 90 entonces
        si_verdadero paneles_gen > demanda entonces
            conmutar_linea(3, 1); // Accionar relés para inyección a red pública
            vender_excedente(paneles_gen - demanda);
        fin_si
    fin_si

    si_verdadero carga < 20 entonces
        si_verdadero es_noche() entonces
            conmutar_linea(2, 0); // Desconectar sectores no esenciales
            preservar_servicios_criticos(); // Asegurar hospital y servidores
        fin_si
    fin_si
```

```
    fin_si  
fin_mientras
```

5. Consideraciones sobre Inteligencia Artificial y Ética

El avance acelerado de los Modelos de Lenguaje Grande (LLMs) ha transformado la ingeniería de software, permitiendo automatizar la escritura mecánica de código y acelerar el aprendizaje conceptual. En el ámbito académico, la cátedra establece lineamientos específicos:

Los estudiantes tienen la libertad de utilizar herramientas de IA para depurar la sintaxis de sus códigos de benchmark o refinar estructuras de control. No obstante, la responsabilidad total y final de la validez semántica e integridad de los códigos recae de forma exclusiva sobre el programador. Se requiere que cada miembro del equipo posea un dominio completo del marco teórico y práctico para justificar el comportamiento de la solución propuesta ante el evaluador.

6. Conclusiones

1. Las decisiones en el diseño de las estructuras morfológicas e intérpretes/compiladores dictan la eficiencia y mantenibilidad del software de manera estructural.
2. El benchmarking demuestra empíricamente que los compilados nativos (Rust y Zig) exhiben rendimientos en tiempo y memoria infinitamente superiores a los entornos interpretados debido a la ausencia de máquinas virtuales complejas y recolección de basura dinámica en el bucle principal.
3. El diseño de lenguajes de dominio específico (DSL) como el Lenguaje L provee una abstracción óptima que reduce los errores en sistemas críticos al limitar las capacidades del programador a las reglas estrictas del negocio.

7. Protocolo de Entrega

La entrega oficial del proyecto se realiza de acuerdo a las directrices vigentes de la coordinación. La entrega formal la realiza el líder de grupo por correo electrónico, conteniendo:

- * **Identificación:** Nombre del grupo, eslogan y listado de participantes.
- * **Recursos:** Enlace al repositorio Git remoto (código, README e informe) y enlace de Drive al video de la defensa.
- * **Cierre:** Nota final en el correo titulada RETO DE FRASES.

RETO DE FRASES (Verificación de Lectura)

Frases clave identificadas en el texto original:

1. *'F: aunque existen diferentes paradigmas siempre nos enamoramos de uno, pero esto no indica que no debemos dominar los demás y aplicarlo según las circunstancias.'*
2. *'F: Rust representa por su seguridad el nuevo estandar para la construccion de kernel Linux.'*

3. 'F: La tokenización en lenguajes es formales diferente a la realizada para los LLM en procesamiento de lenguaje Natural.'
4. 'F: EL benchmarking es una herramienta comparativa interesante para ingeniero de software alto nivel.'
5. 'F: sería ideal hacer una gráfica del benchmarking.'
6. 'F: Un ejercicio creativo para medir su nivel de abstracción en el diseño de un entorno físico con su respectiva comunicación o interfase hombre maquina (lenguaje), su imaginación es el limite!'
7. 'F: bajo las directrices de las Normas APA'

8. Referencias Bibliográficas

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2da ed.). Pearson Educación.
- Ecma International. (2025). ECMAScript 2025 Language Specification. <https://tc39.es/ecma262/>
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). *Introducción a la teoría de autómatas, lenguajes y computación* (3ra ed.). Addison-Wesley.
- Python Software Foundation. (2026). The Python Language Reference (v3.12). <https://docs.python.org/3/reference/>
- Rust Project Developers. (2026). The Rust Reference. <https://doc.rust-lang.org/reference/>
- Zig Software Foundation. (2026). Zig Language Reference. <https://ziglang.org/documentation/>